

RTL Debug & Verification AI Agent — Specification

Version: 1.1

Language: Python 3.11+

Interface: CLI

Status: Validated — Ready for Implementation

Changelog v1.0 → v1.1

All ten open questions from v1.0 have been answered and resolved into the spec.

New sections added: §5.4 HTTP Auth, §6.4 Skill Composition, §7.4 Planner Model Config, §10 Token Budget System, §11 Plugin System. Removed §14 Open Questions.

Table of Contents

1. [Overview](#)
2. [Project Structure](#)
3. [Configuration System](#)
4. [LLM Provider Interface](#)
5. [MCP Server Integration](#)
6. [Skills System](#)
7. [ReAct Loop Engine](#)
8. [Modes](#)
9. [Session & History Management](#)
10. [Token Budget System](#)
11. [Plugin System](#)
12. [CLI Interface](#)
13. [RTL Domain Specifics](#)
14. [Error Handling & Logging](#)
15. [Dependencies](#)

1. Overview

rtl-agent is a command-line AI agent purpose-built for RTL (Register Transfer Level) design debug and verification workflows. It wraps one or more LLM backends behind a uniform interface, orchestrates tool calls via MCP servers (both stdio and HTTP), maintains a composable skill registry the user can extend, and runs an inspectable ReAct (Reason → Act → Observe) loop that the user can tune or override.

Core Design Principles

- **Single config-file truth** — all behaviour is driven by YAML files the user can edit directly.
- **Backend-agnostic** — swapping models or providers requires only a config change, not code changes.
- **Model-locked sessions** — the provider and model are fixed at session creation for reproducibility; the planner may use a separately configured model.
- **Transparent loop** — every step of the ReAct cycle is printed and logged so the user sees what the agent is thinking.
- **Composable skills** — multiple skills can be active simultaneously; their prompts and tool preferences are merged.
- **User-steerable** — skills, modes, the ReAct loop, and plugins are all data or Python extension points; the user never needs to touch core agent code.
- **Local-first history** — sessions are stored entirely on disk under `sessions/` ; all artifacts live inside the session directory.
- **Budget-aware** — a configurable token budget is tracked live and displayed during a session; the agent stops when the budget is exhausted.

2. Project Structure

```
rtl-agent/
├── agent/                                # Core agent logic
│   ├── __init__.py
│   ├── core.py                          # Agent orchestrator – top-level run loop
│   ├── react_loop.py                    # ReAct engine (Thought → Action → Observatio
│   └── planner.py                       # Optional multi-step planning layer (own mod
```

```

|   └─ context.py                # Context/state object passed through the loop
|
| └─ llm/                        # LLM provider abstraction
|   └─ __init__.py
|   └─ base.py                  # Abstract LLMProvider base class
|   └─ anthropic_provider.py    # Claude (all models)
|   └─ openai_provider.py       # OpenAI (GPT-4o, o1, etc.)
|   └─ gemini_provider.py       # Google Gemini
|   └─ ollama_provider.py       # Local Ollama models
|   └─ provider_factory.py      # Resolves provider from config
|
| └─ mcp/                        # MCP server connection layer
|   └─ __init__.py
|   └─ base.py                  # Abstract MCPClient base class
|   └─ stdio_client.py          # stdio transport (subprocess + JSON-RPC 2.0)
|   └─ http_client.py           # HTTP/SSE transport
|   └─ auth/                    # HTTP auth strategies
|     └─ __init__.py
|     └─ bearer.py              # Bearer token auth
|     └─ api_key.py             # API key auth (header or query param)
|     └─ oauth.py               # OAuth 2.0 client credentials flow
|   └─ registry.py              # Discovers and holds connected servers
|   └─ tool_dispatcher.py       # Routes tool calls to the right server
|
| └─ skills/                     # Skills registry
|   └─ __init__.py
|   └─ registry.py              # Loads, validates, exposes, and composes skills
|   └─ composer.py              # Merges multiple active skills into one context
|   └─ builtin/                 # Built-in RTL skills (YAML)
|     └─ rtl_debug.yaml
|     └─ verification_plan.yaml
|     └─ testbench_writer.yaml
|     └─ spec_writer.yaml
|     └─ lint_checker.yaml
|     └─ waveform_analyzer.yaml
|
| └─ modes/                      # Operating modes
|   └─ __init__.py
|   └─ registry.py              # Loads mode definitions
|   └─ builtin/                 # Built-in mode definitions (YAML)
|     └─ plan.yaml
|     └─ code.yaml
|     └─ spec.yaml
|     └─ verify.yaml
|     └─ debug.yaml
|     └─ review.yaml
|

```



```
|   └─ agent.log
|
|─ tests/
|   └─ test_llm/
|   └─ test_mcp/
|   └─ test_skills/
|   └─ test_react/
|   └─ test_budget/
|   └─ test_plugins/
|   └─ test_cli/
|
|─ .env                                # API keys and secrets (auto-loaded at startu
|─ .env.example                       # Template showing all supported env vars
|─ pyproject.toml
|─ requirements.txt
|─ README.md
```

3. Configuration System

All configuration lives in `config/`. Files are YAML. The agent loads and validates them at startup via Pydantic models, merging everything into a single settings object. Users can edit files directly or use `rtl-agent config` commands.

3.1 `.env` Loading

The agent automatically loads `.env` from the **project root directory** at startup using `python-dotenv`. No flag is required. A `.env.example` is shipped with the repo documenting all supported variables. If `.env` is not found the agent silently continues, relying on environment variables already present in the shell.

```
# .env.example
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
GEMINI_API_KEY=AIza...
SIM_TOKEN=...
CODEBASE_CLIENT_ID=...
CODEBASE_CLIENT_SECRET=...
COVERAGE_API_KEY=...
SLACK_WEBHOOK=https://hooks.slack.com/...
```

Env var interpolation in YAML: any value matching `${VAR_NAME}` is substituted at load time. If a variable is missing the agent logs a warning and sets the value to `null`; it will not

crash until that provider or server is actually used.

An additional env file can be provided via `--env-file <path>` in `rtl-agent run`. It is merged on top of the auto-loaded `.env`.

3.2 config/settings.yaml

```
agent:
  name: "rtl-agent"
  default_provider: "anthropic"           # anthropic | openai | gemini | ollama
  default_model: "claude-sonnet-4-20250514"
  default_mode: "debug"
  default_skills: []                     # Active skills by default (list, composa
  max_react_iterations: 15
  show_thinking: true                   # Print Thought steps to terminal
  show_tool_calls: true
  temperature: 0.2
  stream_output: true

planner:
  enabled: false                       # Planners are opt-in per mode
  provider: "anthropic"                # Can differ from the main session model
  model: "claude-opus-4-20250514"
  temperature: 0.1

budget:
  enabled: true
  max_input_tokens: 500000             # Cumulative input token cap for the sessi
  max_output_tokens: 100000           # Cumulative output token cap for the sess
  warn_at_percent: 80                 # Show warning when budget is 80% consumed
  hard_stop: true                     # Stop when exhausted (false = warn only)

sessions:
  storage_dir: "./sessions"
  auto_save: true

logging:
  level: "INFO"                       # DEBUG | INFO | WARNING | ERROR
  log_file: "./logs/agent.log"
  log_react_steps: true

plugins:
  enabled: true
  plugin_dir: "./plugins_user"
```

```
ui:
  theme: "dark"                                # dark | light
  show_token_count: true
  show_budget_bar: true                        # Show live budget bar after each turn
```

3.3 config/providers.yaml

```
providers:
  anthropic:
    api_key: "${ANTHROPIC_API_KEY}"
    default_model: "claude-sonnet-4-20250514"
    available_models:
      - "claude-opus-4-20250514"
      - "claude-sonnet-4-20250514"
      - "claude-haiku-4-5-20251001"
    base_url: null                            # Override for proxies
    timeout: 120

  openai:
    api_key: "${OPENAI_API_KEY}"
    default_model: "gpt-4o"
    available_models:
      - "gpt-4o"
      - "gpt-4o-mini"
      - "o1"
      - "o1-mini"
    base_url: null
    timeout: 120

  gemini:
    api_key: "${GEMINI_API_KEY}"
    default_model: "gemini-2.0-flash"
    available_models:
      - "gemini-2.5-pro"
      - "gemini-2.0-flash"
    timeout: 120

  ollama:
    base_url: "http://localhost:11434"
    default_model: "llama3.2"
    available_models: []                      # Auto-discovered from Ollama at startup
    timeout: 300
```

3.4 config/mcp_servers.yaml

```
mcp_servers:
- name: "waveform-tools"
  description: "VCD/FSDB waveform inspection and query tools"
  transport: "stdio"
  command: "python"
  args: ["-m", "waveform_mcp"]
  env:
    WAVE_DIR: "/data/waves"
  enabled: true
  auth: null # Auth not applicable for stdio transport

- name: "rtl-linter"
  description: "Verilator and svlint integration"
  transport: "stdio"
  command: "/usr/local/bin/rtl_linter_mcp"
  args: []
  enabled: true
  auth: null

- name: "sim-runner"
  description: "Simulation runner - HTTP MCP with bearer token"
  transport: "http"
  url: "http://localhost:8080/mcp"
  enabled: true
  auth:
    type: "bearer"
    token: "${SIM_TOKEN}"

- name: "codebase"
  description: "RTL codebase search - HTTP MCP with OAuth 2.0"
  transport: "http"
  url: "https://internal.corp/mcp/codebase"
  enabled: false
  auth:
    type: "oauth"
    token_url: "https://internal.corp/oauth/token"
    client_id: "${CODEBASE_CLIENT_ID}"
    client_secret: "${CODEBASE_CLIENT_SECRET}"
    scopes: ["read:rtl", "read:sim"]

- name: "coverage-db"
  description: "Coverage database query - HTTP MCP with API key"
  transport: "http"
  url: "https://internal.corp/mcp/coverage"
```



```
enabled: false
auth:
  type: "api_key"
  header: "X-API-Key"          # Use "query_param" field for URL param aut
  value: "${COVERAGE_API_KEY}"
```

3.5 config/plugins.yaml

```
plugins:
- name: "vcd_preprocessor"
  path: "./plugins_user/vcd_preprocessor.py"
  enabled: true
  config:
    max_signals: 500

- name: "slack_notifier"
  path: "./plugins_user/slack_notifier.py"
  enabled: false
  config:
    webhook_url: "${SLACK_WEBHOOK}"
```

4. LLM Provider Interface

4.1 Abstract Base Class

```
# llm/base.py
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import AsyncIterator

@dataclass
class Message:
    role: str          # "system" | "user" | "assistant" | "tool"
    content: str | list # str for text; list for multi-part (tool results)
    tool_call_id: str | None = None
    tool_calls: list | None = None
    react_step: str | None = None # "thought" | "action" | "observation"

@dataclass
class LLMResponse:
    content: str
```

```

    tool_calls: list[dict]
    input_tokens: int
    output_tokens: int
    stop_reason: str          # "end_turn" | "tool_use" | "max_tokens"

class LLMProvider(ABC):
    @abstractmethod
    async def complete(
        self,
        messages: list[Message],
        tools: list[dict],          # OpenAI-format JSON Schema tool definitions
        system: str | None,
        temperature: float,
        stream: bool,
    ) -> LLMResponse: ...

    @abstractmethod
    async def stream_complete(
        self,
        messages: list[Message],
        tools: list[dict],
        system: str | None,
        temperature: float,
    ) -> AsyncIterator[str]: ...

    @abstractmethod
    def list_models(self) -> list[str]: ...

```

4.2 Provider Normalisation

Each provider adapter translates its native API format to/from `Message` and `LLMResponse`. The ReAct loop never addresses a provider directly. Tool schemas are always passed in OpenAI-compatible JSON Schema format; adapters convert as required (Anthropic's tool definition format differs; Gemini uses function declarations).

4.3 Session Model Lock

The provider and model are recorded in `metadata.json` at session creation and cannot be changed mid-session. If the user types `/model` during a session, the CLI shows a clear message explaining the lock and offers to open a new session with the requested model. This ensures full reproducibility when resuming or auditing a session.

4.4 Streaming

When `stream_output: true`, the CLI prints tokens as they arrive via `stream_complete`. The

fully assembled response is stored in `messages.jsonl` after streaming completes.

5. MCP Server Integration

5.1 Transport Support

Transport	Mechanism	Typical Use
<code>stdio</code>	Subprocess; JSON-RPC 2.0 over <code>stdin/stdout</code>	Local tools — linters, simulators, file tools
<code>http</code>	HTTP POST; SSE for streaming responses	Remote services, team infrastructure

5.2 Abstract MCP Client

```
# mcp/base.py
from abc import ABC, abstractmethod

class MCPCClient(ABC):
    name: str
    description: str

    @abstractmethod
    async def connect(self) -> None: ...

    @abstractmethod
    async def disconnect(self) -> None: ...

    @abstractmethod
    async def list_tools(self) -> list[dict]: ...    # Returns JSON Schema tool de

    @abstractmethod
    async def call_tool(self, tool_name: str, arguments: dict) -> dict: ...

    @abstractmethod
    async def health_check(self) -> bool: ...
```

5.3 MCP Registry

`mcp/registry.py` reads `mcp_servers.yaml` , instantiates the correct client class for each enabled server, runs `connect()` , and builds a unified deduplicated tool list injected into

every LLM call. Tool names are namespaced `<server-name>__<tool-name>` to prevent collisions across servers.

5.4 HTTP Authentication

Three auth strategies are supported, selected by `auth.type` in `mcp_servers.yaml`. Auth credentials support `${ENV_VAR}` interpolation and are never written to logs.

Bearer Token (`type: bearer`)

Adds `Authorization: Bearer <token>` to every HTTP request.

API Key (`type: api_key`)

Adds the key as a custom HTTP header (configurable via `header` field, e.g. `X-API-Key`) or as a URL query parameter (use `query_param` field instead). Value supports env var interpolation.

OAuth 2.0 Client Credentials (`type: oauth`)

Fetches a short-lived access token from `token_url` using `client_id` and `client_secret` before the first request. The token is cached in memory and refreshed automatically when it expires (using `expires_in` from the token response). Per-server `scopes` are configurable. On token refresh failure the server is disabled and the user is notified.

5.5 Tool Dispatcher

`mcp/tool_dispatcher.py` receives a tool call from the LLM, strips the server prefix from the namespaced tool name, looks up the correct `MCPClient`, calls `call_tool(name, args)`, and returns the result as a tool-result message appended to the conversation.

5.6 Waveform File Handling

Waveform files (VCD, FSDB, etc.) are handled entirely by MCP tools. The agent passes file paths as string arguments to the relevant MCP tool — it does not read or process binary waveform files directly. When a user references a waveform path in the CLI, that path is included in the message text; the LLM extracts it and passes it to the appropriate MCP tool.

5.7 Lifecycle

- **Startup:** connect all enabled servers; log a warning for any that fail (non-fatal; failed server's tools are absent from the tool list).
- **Shutdown:** gracefully disconnect all servers.
- **Hot-reload:** `rtl-agent mcp reload` reconnects all servers without restarting the process.

6. Skills System

A **skill** is a named, reusable instruction block that scopes the agent's behaviour for a specific RTL task — what to focus on, which tools to prefer, and optionally a few-shot example. Skills are YAML files in `config/skills/` (user-defined) or `agent/skills/builtin/` (shipped defaults).

6.1 Skill Schema

```
# config/skills/rtl_debug.yaml
name: "rtl_debug"
version: "1.0"
description: "Debug RTL simulation failures — find root cause and suggest fix"

system_prompt: |
  You are an expert RTL debug engineer with deep knowledge of SystemVerilog,
  Verilog, VHDL, and common EDA tools (VCS, Questa, Xcelium, Verilator).
  Identify the root cause of a simulation failure and propose a minimal, correct
  fix. Reason step by step. Always cite signal names, timestamps, and file lines.

preferred_tools:
  - "waveform-tools__*"
  - "rtl-linter__*"
  - "codebase__search"

suggested_mode: "debug"

few_shot_examples:
  - user: "The sim fails at 500ns with a X-prop on signal ready"
    assistant: |
      Thought: I need to find what drives 'ready' and trace back the X source.
      Action: waveform-tools__query_signal(name="ready", time_range=[490, 510])

tags: ["debug", "simulation", "x-prop"]
```

6.2 Built-in Skills

Skill	Purpose
rtl_debug	Root-cause a simulation failure
verification_plan	Generate a structured verification plan from a spec

testbench_writer	Write SystemVerilog/UVM testbench stubs
spec_writer	Extract or author a design specification
lint_checker	Run and interpret lint results
waveform_analyzer	Analyse waveform dumps, find timing bugs

6.3 Management CLI

```

rtl-agent skill list           # Show all skills (built-in + user-defined)
rtl-agent skill show <name>   # Print skill YAML
rtl-agent skill edit <name>   # Open in $EDITOR
rtl-agent skill create <name> # Create new skill from template
rtl-agent skill delete <name> # Remove a user-defined skill

```

6.4 Skill Composition

Multiple skills can be active simultaneously. When two or more skills are active, `skills/composer.py` merges them before injecting the combined block into the LLM system prompt.

Composition rules:

Field	Merge Strategy
system_prompt	Concatenated in activation order, separated by a blank line
preferred_tools	Unioned; duplicates removed
few_shot_examples	Concatenated in activation order
suggested_mode	First active skill's suggestion wins; later ones are ignored

Skills are activated at session start (from `settings.yaml` → `agent.default_skills` and the mode's `preferred_skills`) or toggled mid-session via `/skill add <name>` and `/skill remove <name>`.

```

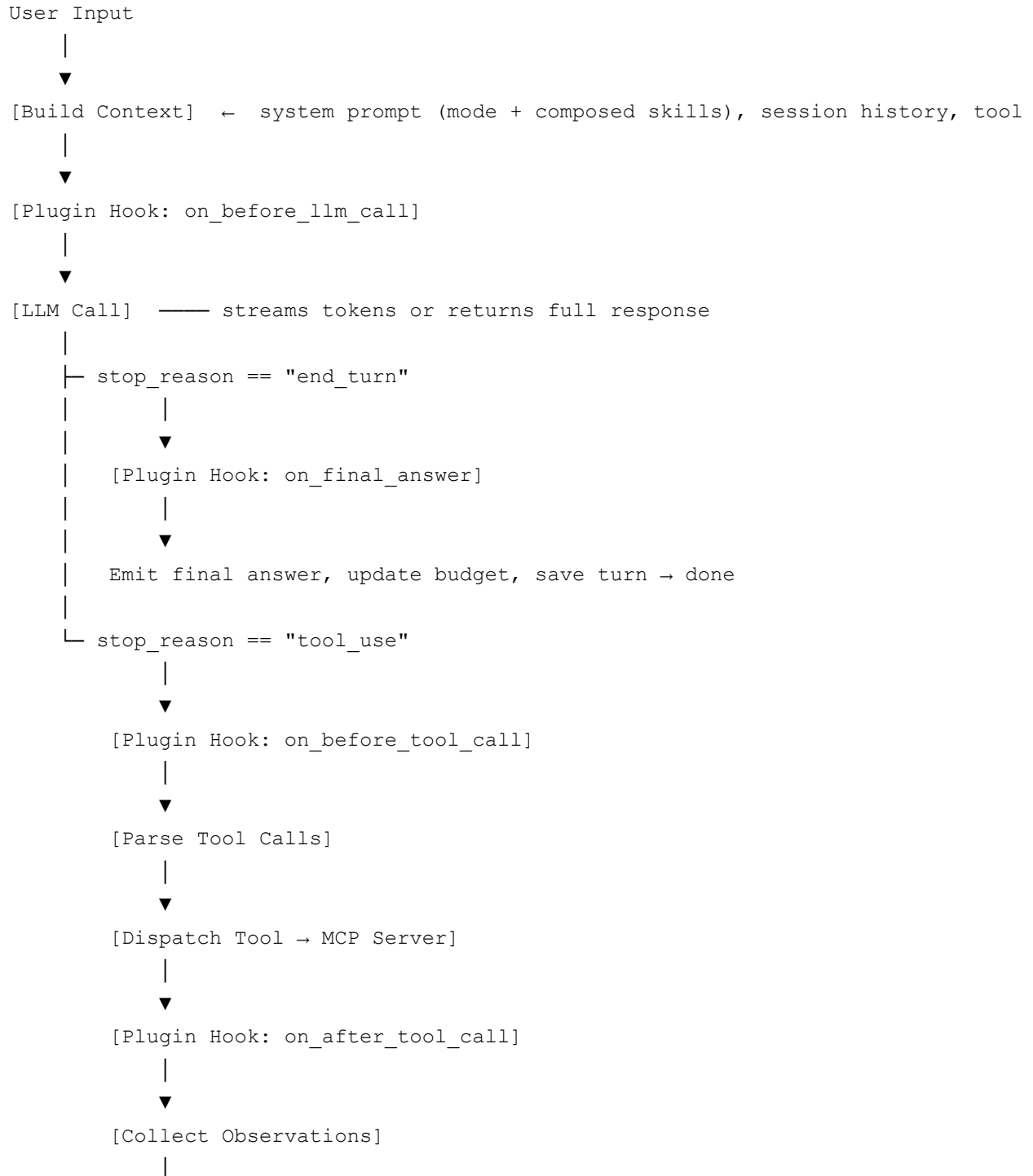
# settings.yaml example
agent:
  default_skills:
    - "rtl_debug"
    - "waveform_analyzer"

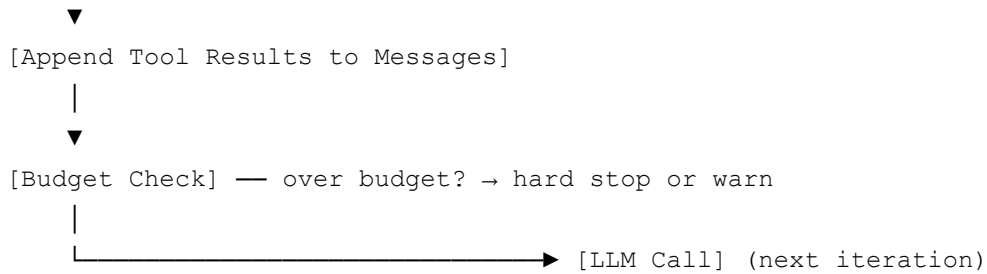
```

7. ReAct Loop Engine

The ReAct loop is the agent's core reasoning engine. It runs **Thought → Action → Observation** cycles until the LLM produces a final answer, the iteration limit is reached, or the token budget is exhausted.

7.1 Loop Flow





7.2 Loop Configuration

Behaviour is driven by `settings.yaml` and can be overridden per mode:

```

react_loop:
  max_iterations: 15
  show_thought: true          # Print Thought text to terminal (dimmed grey)
  show_action: true           # Print tool name + args (cyan)
  show_observation: true      # Print tool result summary (yellow)
  abort_on_error: false       # false = inject error as Observation and continue
  iteration_warning: 10       # Warn when this many iterations remain
  
```

7.3 Hook Points

The `ReActLoop` class exposes hook methods called at each stage. Plugins (§11) implement these; all registered plugins are called in registration order at each hook.

Hook	Trigger
<code>on_session_start(context)</code>	Once, after session is initialised
<code>on_before_llm_call(messages, tools)</code>	Before every LLM API call
<code>on_after_llm_call(response)</code>	After every LLM API call
<code>on_before_tool_call(tool_name, arguments)</code>	Before dispatching each MCP tool call
<code>on_after_tool_call(tool_name, arguments, result)</code>	After receiving each MCP tool result
<code>on_final_answer(content)</code>	When the LLM emits a final answer
<code>on_session_end(context)</code>	Once, when the session ends or is interrupted
<code>on_budget_warning(budget_status)</code>	When the warn threshold is crossed


```
on_budget_exhausted(budget_status)
```

When the hard budget limit is reached

7.4 Planner (Optional Pre-step)

When a mode sets `use_planner: true`, a planning pass runs before the main loop using its **own separately configured model** (from `settings.yaml` → `planner:`). The planner produces a numbered step list stored in `context.plan`. The main ReAct loop works through steps sequentially. If the mode sets `plan_approval: true`, the CLI displays the plan and waits for the user to confirm or edit it before execution begins.

```
# settings.yaml
planner:
  provider: "anthropic"
  model: "claude-opus-4-20250514"    # Larger model for planning
  temperature: 0.1

# The main session model can be faster/cheaper:
agent:
  default_model: "claude-sonnet-4-20250514"
```

Planner token usage is counted against the same session budget.

8. Modes

A **mode** frames the agent's high-level goal, tunes ReAct loop parameters, enables or disables the planner, and declares preferred skills. Modes are YAML files in `config/modes/` (user) or `agent/modes/builtin/` (shipped defaults).

8.1 Mode Schema

```
# agent/modes/builtin/debug.yaml
name: "debug"
description: "Root cause a failing simulation or synthesis issue"

system_prompt_suffix: |
  Focus on identifying the exact failure point before proposing any fix.
  Do not speculate about multiple causes simultaneously — narrow down first.

preferred_skills:
  - "rtl_debug"
  - "waveform_analyzer"
```

```
use_planner: false
plan_approval: false

react_loop:
  max_iterations: 20
  show_thought: true
  abort_on_error: false

output_format: "markdown"           # markdown | plain | json
```

8.2 Built-in Modes

Mode	Description	Planner
debug	Trace and resolve simulation failures	No
plan	Decompose a task into an ordered action plan	Yes
code	Write, modify, or refactor RTL or TB code	No
spec	Extract or author design specifications	No
verify	Create verification plans and coverage models	Yes
review	Review RTL for correctness, style, and coverage	No

8.3 Mode Management CLI

```
rtl-agent mode list
rtl-agent mode show <name>
rtl-agent mode edit <name>
rtl-agent mode create <name>
```

9. Session & History Management

9.1 Session Structure

All session data — messages, artifacts, metadata — lives exclusively inside `sessions/<session-id>/`. Nothing is written outside this directory during a session.

```
sessions/
```

```

└─ 20260505-143201-a3f2/
    ├── metadata.json      # Session ID, model lock, active skills, budget total
    ├── messages.jsonl     # One JSON object per line; full message history
    └─ artifacts/          # Every file the agent produces
        ├── dut_fix.sv
        └─ verification_plan.md

```

metadata.json schema:

```

{
  "session_id": "20260505-143201-a3f2",
  "created_at": "2026-05-05T14:32:01Z",
  "last_updated": "2026-05-05T15:10:45Z",
  "provider": "anthropic",
  "model": "claude-sonnet-4-20250514",
  "model_locked": true,
  "planner_provider": "anthropic",
  "planner_model": "claude-opus-4-20250514",
  "mode": "debug",
  "active_skills": ["rtl_debug", "waveform_analyzer"],
  "title": "X-prop on ready signal at 500ns",
  "tags": [],
  "message_count": 42,
  "budget": {
    "max_input_tokens": 500000,
    "max_output_tokens": 100000,
    "used_input_tokens": 18400,
    "used_output_tokens": 6200
  }
}

```

messages.jsonl : One JSON line per turn. Includes `role`, `content`, `tool_calls`, `tool_results`, `input_tokens`, `output_tokens`, `timestamp`, and `react_step` (`thought` | `action` | `observation`).

9.2 Session Commands

<code>rtl-agent session list</code>	# Table of recent sessions
<code>rtl-agent session list --filter "x-prop"</code>	# Search by title or tag
<code>rtl-agent session show <id></code>	# Print metadata + budget summary
<code>rtl-agent session resume <id></code>	# Load history, continue in CLI
<code>rtl-agent session export <id> --format md</code>	# Export to markdown / html / json
<code>rtl-agent session delete <id></code>	
<code>rtl-agent session tag <id> --add "critical"</code>	
<code>rtl-agent session artifacts <id></code>	# List artifacts with file sizes

9.3 Auto-save

When `auto_save: true`, every completed message turn is written to `messages.jsonl` immediately after the LLM responds. `metadata.json` is updated with current token and budget totals after each turn. No data is lost if the agent crashes.

10. Token Budget System

10.1 Purpose

A session-level token budget prevents runaway costs and gives the user live visibility into consumption during long debug or verification sessions.

10.2 Configuration

Configured in `settings.yaml` → `budget: .` Overridable per session via CLI flags.

```
budget:
  enabled: true
  max_input_tokens: 500000
  max_output_tokens: 100000
  warn_at_percent: 80
  hard_stop: true
```

Per-session override:

```
rtl-agent run --budget-input 200000 --budget-output 50000
```

10.3 Tracking

`budget/tracker.py` maintains a running total of `(input_tokens, output_tokens)` using counts from every `LLMResponse`. Planner calls count against the same budget. MCP tool calls are not LLM calls and are not counted.

10.4 Display

When `show_budget_bar: true`, a compact budget line is shown after each assistant turn:

```
[Budget ██████████░░░░ 58% in: 18.4k / 500k out: 6.2k / 100k]
```

The bar turns yellow at `warn_at_percent` and red when exhausted. Users can check the budget at any time with `/budget`.

10.5 Hard Stop Behaviour

When `hard_stop: true` and the budget is exhausted:

- The `on_budget_exhausted` plugin hook fires.
- The current ReAct iteration completes (no mid-stream cutoff).
- A budget exhaustion panel is printed with final totals.
- The session is saved.
- The agent exits the run loop. The user can start a new session.

When `hard_stop: false`, the agent prints a warning and continues; the user accepts responsibility for further costs.

10.6 Budget CLI

```
rtl-agent budget status # Show budget for last session
rtl-agent budget set --input 300000 --output 80000 # Update defaults in settings
```

11. Plugin System

Plugins extend the agent at defined hook points (§7.3) without modifying core code. They are Python files placed in `plugins_user/` (auto-discovered) or explicitly registered in `config/plugins.yaml`.

11.1 Plugin Base Class

```
# plugins/base.py
from abc import ABC
from typing import Any

class Plugin(ABC):
    """Base class for all rtl-agent plugins. Override only the hooks you need."""

    name: str = "unnamed_plugin"
    version: str = "1.0"
    description: str = ""
```

```

def __init__(self, config: dict) -> None: ...

# — Lifecycle —————
def on_session_start(self, context: Any) -> None: ...
def on_session_end(self, context: Any) -> None: ...

# — LLM call hooks —————
def on_before_llm_call(self, messages: list, tools: list) -> None: ...
def on_after_llm_call(self, response: Any) -> None: ...

# — Tool call hooks —————
def on_before_tool_call(self, tool_name: str, arguments: dict) -> None: ...
def on_after_tool_call(
    self, tool_name: str, arguments: dict, result: dict
) -> None: ...

# — Answer hook —————
def on_final_answer(self, content: str) -> None: ...

# — Budget hooks —————
def on_budget_warning(self, budget_status: dict) -> None: ...
def on_budget_exhausted(self, budget_status: dict) -> None: ...

```

All hook methods have default no-op implementations. Plugins override only what they need.

11.2 Plugin Loader

`plugins/loader.py` at startup:

1. Reads `config/plugins.yaml` for explicitly registered plugins.
2. Scans `plugins_user/` for any `.py` files not yet registered (auto-discovery).
3. Imports each enabled plugin module, finds the `Plugin` subclass, instantiates it with its `config dict`.
4. Registers all instances in a global plugin registry in the order defined in `plugins.yaml`; auto-discovered plugins come last.

Plugin load errors are non-fatal — a warning is logged and that plugin is skipped.

11.3 Example Plugin — Slack Notifier

```
# plugins_user/slack_notifier.py
```

```

import httpx
from plugins.base import Plugin

class SlackNotifierPlugin(Plugin):
    name = "slack_notifier"
    description = "Posts a Slack message when the agent produces a final answer"

    def __init__(self, config: dict):
        self.webhook_url = config.get("webhook_url", "")

    def on_final_answer(self, content: str) -> None:
        if not self.webhook_url:
            return
        httpx.post(self.webhook_url, json={"text": f"rtl-agent: {content[:200]}"})

```

11.4 Example Plugin — VCD Preprocessor

```

# plugins_user/vcd_preprocessor.py
from pathlib import Path
from plugins.base import Plugin

class VCDPreprocessorPlugin(Plugin):
    name = "vcd_preprocessor"
    description = "Pre-processes oversized VCD file paths before MCP tool calls"

    def __init__(self, config: dict):
        self.max_signals = config.get("max_signals", 500)

    def on_before_tool_call(self, tool_name: str, arguments: dict) -> None:
        if "waveform" not in tool_name:
            return
        path = arguments.get("file_path", "")
        if path and Path(path).suffix in {".vcd", ".fsdb"}:
            # Pre-process / filter signals here before the MCP call
            ...

```

11.5 Plugin Management CLI

rtl-agent plugin list	# All plugins and their enabled status
rtl-agent plugin show <name>	# Print plugin metadata and config
rtl-agent plugin enable <name>	
rtl-agent plugin disable <name>	
rtl-agent plugin reload	# Hot-reload all plugins without restarti

12. CLI Interface

The CLI is built with **Typer** and **Rich**.

12.1 Top-Level Commands

```
rtl-agent --help
```

Commands:

run	Start or resume an agent session (main interactive loop)
session	Manage saved sessions
skill	Manage skills
mode	Manage modes
mcp	Manage MCP server connections
plugin	Manage plugins
budget	View and configure token budgets
config	View and edit configuration
model	List available models across providers

12.2 `rtl-agent run` — Main Interactive Loop

```
rtl-agent run [OPTIONS]
```

Options:

<code>--provider TEXT</code>	Override default LLM provider	<code>[env: RTL_PROVIDER]</code>
<code>--model TEXT</code>	Override default model	<code>[env: RTL_MODEL]</code>
<code>--mode TEXT</code>	Override default mode	<code>[env: RTL_MODE]</code>
<code>--skills TEXT</code>	Comma-separated skill names	<code>[env: RTL_SKILLS]</code>
<code>--session TEXT</code>	Resume an existing session ID	
<code>--title TEXT</code>	Title for this new session	
<code>--budget-input INT</code>	Override max input token budget	
<code>--budget-output INT</code>	Override max output token budget	
<code>--env-file PATH</code>	Load an additional .env file	
<code>--no-stream</code>	Disable streaming output	
<code>--quiet</code>	Suppress ReAct step printing	
<code>--help</code>		

In-session slash commands:

<code>/help</code>	Show all in-session commands
<code>/mode <name></code>	Switch mode (applies from next turn)
<code>/skill add <name></code>	Add a skill to the active composition
<code>/skill remove <name></code>	Remove a skill from the active composition

/skill list	Show currently active skills
/plan	Run the planner on the current task
/tools	List all available MCP tools
/budget	Show current budget consumption and bar
/history	Show message count and token totals
/save	Force-save the current session
/export	Export current session to markdown in artifacts/
/clear	Clear context (start a new sub-task)
/quit	Save session and exit

12.3 rtl-agent mcp Subcommands

rtl-agent mcp list	# All configured servers + status
rtl-agent mcp status	# Health-check all connected servers
rtl-agent mcp add	# Interactive wizard to add a server
rtl-agent mcp remove <name>	
rtl-agent mcp enable <name>	
rtl-agent mcp disable <name>	
rtl-agent mcp reload	# Reconnect all servers (hot reload)
rtl-agent mcp tools <name>	# List tools exposed by a server

12.4 rtl-agent config Subcommands

rtl-agent config show	# Print current effective merged conf
rtl-agent config edit	# Open settings.yaml in \$EDITOR
rtl-agent config set <key> <value>	# Set dotted-path key
rtl-agent config get <key>	
rtl-agent config validate	# Validate all config files
rtl-agent config reset	# Reset to defaults (with confirmatio

12.5 rtl-agent model Subcommands

rtl-agent model list	# All models across all providers
rtl-agent model list --provider anthropic	
rtl-agent model set <provider/model>	# Update default in settings.yaml

12.6 Terminal Output Conventions

Output type	Style	Prefix
Thought step	Dimmed grey	[Thought]

Action step	Cyan	[Action → tool_name]
Observation step	Yellow	[Observation]
Final answer	Full-width Rich panel, white	—
Plugin event	Dimmed blue	[Plugin: name]
Warning	Yellow	[Warning]
Error	Red	[Error]
Budget bar	Inline after each turn	[Budget  58% in: 18.4k/500k out: 6.2k/100k]
Token count	Dimmed, after final answer	(in: 1,240 / out: 380 tokens this turn)

13. RTL Domain Specifics

13.1 Project Context Auto-detection

When the agent starts in a directory containing an RTL project, it scans for known markers and injects detected context into the system prompt. Detection is opportunistic and non-blocking.

Detected items:

- Top-level module name (from `*.sv` / `*.v` scan or `config/project.yaml`)
- Simulator type (`VCS` , `Questa` , `Xcelium` , `Verilator`) via `Makefile` or `sim_setup.yaml`
- Active regression suite name if configured

Override via `config/project.yaml` :

```
project:
  name: "my_soc"
  top_module: "my_soc_top"
  simulator: "vcs"
  rtl_root: "./rtl"
  tb_root: "./tb"
```

```
sim_script: "./run_sim.sh"
```

13.2 Artifact Types

All artifacts are saved inside `sessions/<id>/artifacts/` . The terminal prints the artifact path when a file is written.

Type	Extension(s)
SystemVerilog RTL	.sv
Verilog RTL	.v
VHDL	.vhd
Testbench	.sv , .v
UVM sequences / agents	.sv
Verification Plan	.md
Design Spec	.md
TCL scripts	.tcl
Makefiles	Makefile
Coverage reports	.md , .html
Session export	.md , .html , .json

14. Error Handling & Logging

14.1 Failure Modes

Failure	Behaviour
LLM API error (rate limit, timeout)	Retry with exponential back-off (3 attempts, max 60 s), then surface to user
MCP tool call failure	Inject error text as Observation; LLM decides to retry or abandon

MCP server unreachable at startup	Log warning, continue; that server's tools are absent
OAuth token refresh failure	Log error, disable that server, notify user
Config validation failure	Hard exit with clear error message, file name, and line number
Plugin load error	Log warning, skip that plugin, continue
Iteration limit reached	Print warning, prompt user: continue N more iterations or stop
Context length exceeded	Warn user, offer to summarise and trim oldest messages
Budget exhausted (<code>hard_stop: true</code>)	Save session, print totals, exit run loop

14.2 Logging

All log entries go to `logs/agent.log` as structured JSON (one object per line). Fields: `timestamp`, `level`, `component`, `message`, `extra`. Auth credentials and API keys are never logged. ReAct loop steps are logged at `DEBUG` with full content. Plugin events are logged at `INFO`.

15. Dependencies

```
[project]
requires-python = ">=3.11"

dependencies = [
    # CLI & terminal
    "typer[all]>=0.12",
    "rich>=13",

    # Config & data models
    "pydantic>=2",
    "pyyaml>=6",
    "python-dotenv>=1",

    # Async HTTP (MCP HTTP transport, OAuth token fetch)
    "httpx>=0.27",
```

```
# LLM provider SDKs
"anthropic>=0.40",
"openai>=1.50",
"google-generativeai>=0.8",
"ollama>=0.3",

# Async runtime
"anyio>=4",

# Session message storage
"jsonlines>=4",
]

[project.optional-dependencies]
dev = [
    "pytest>=8",
    "pytest-asyncio>=0.23",
    "ruff>=0.4",
    "mypy>=1.10",
]
```

End of Specification v1.1 — All decisions validated. Ready for implementation.